

ClickHouse在eBay大规模版本升级

实践

从v21到v24的无缝迁移

eBay | 刘秋阳 | May, 2026

目录

01

eBay与NuColumnar简介

- 平台定位与核心能力
- 架构概览与客户规模

02

升级背景与目标

- 当前版本(v21)的瓶颈与挑战
- 目标版本(v24)的核心新特性与目标

03

核心迁移策略

- 为什么不选择原地升级？
- 逐分片Dual Consume策略与关键原则

04

实施细节与保障

- 分片迁移详细流程拆解
- 数据完整性三层保障与回滚预案

05

挑战、成果与总结

- 规模化挑战与解决方案
- 升级成果量化展示与经验思考

eBay与NuColumnar简介

NuColumnar 是 eBay内基于ClickHouse构建的实时OLAP平台。



集群管理

基于 Coordinator Control Plane 管控, 实现 Schema 管理、集群扩缩容与健康监控的自动化



实时数据写入

- 超高 ingestion 吞吐量 : > 1M rows/s
- Kafka + Block Ingest



离线分区替换

- 支持 Hadoop 生态高效批量数据导入
- Airflow-driven offline partition replacement



实时查询服务

- JDBC



产品里程碑

2020年6月上线

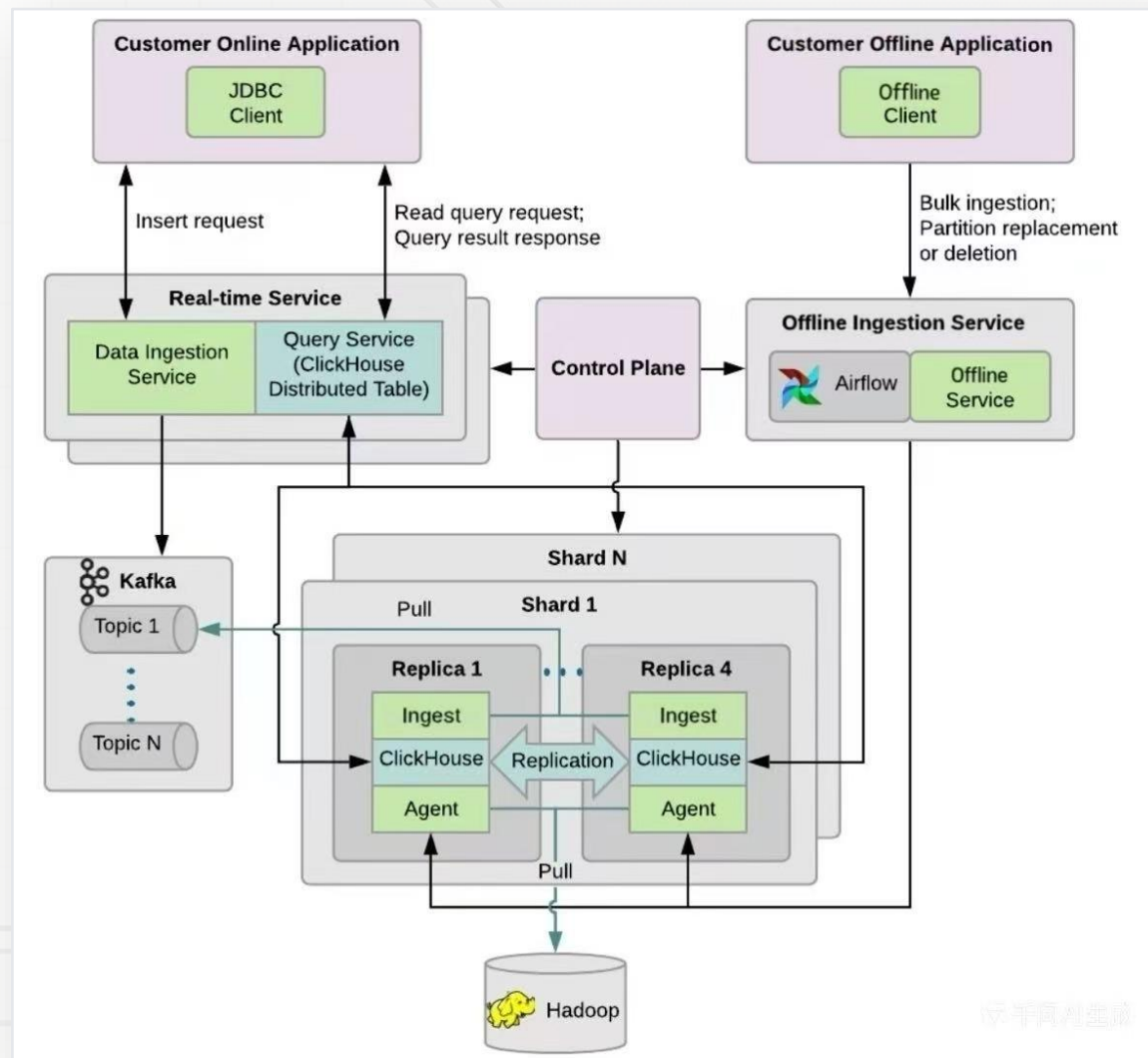


核心客户规模 (Shards)

核心业务 Ads

250 Shards

NuColumnar 架构



升级背景：为何必须升级？

当前版本: ClickHouse v21.8.3.44-lts

⚠️ 功能限制

- 缺乏对复杂JSON数据类型的高效解析与查询支持
- 缺少Projection（投影）功能，无法实现自动预聚合优化
- Materialized View（物化视图）功能受限，不支持 JOIN 关联操作

🛑 技术瓶颈 (ZooKeeper Bottleneck)

- Ads keypace 250 shards, 5 zookeeper clusters
高吞吐场景下，单集群需承载50 Shard，已达性能极限，导致元数据同步频繁异常

目标版本: ClickHouse v24.4.1.2088-stable

🚀 强大的新特性支持

- Materialized View with JOINS：支持复杂关联查询的高效预计算，大幅降低查询延迟
- Projection & Data Skipping：自动选择最优预聚合路径，显著加速大范围数据扫描
- 大幅增强了对JSON与嵌套数据类型的原生处理能力
- 升级query optimizer

升级的核心目标

核心目标：实现从 v21 到 v24 的无缝升级，对客户业务**透明无感知**



服务连续性

High Availability

- 读操作：**零停机时间 (Zero Downtime)**
- 实时写入：仅允许**分钟级别的数据延迟**，对上游生产者无任何影响。



数据完整性

Data Integrity

- **零数据丢失**：全链路校验，确保所有历史与增量数据均被正确迁移，无遗漏。
- **零数据重复**：严格的幂等性设计，确保数据在迁移过程中不被重复消费与处理。



系统可扩展性

Scalability

- **架构瓶颈突破**：
缓解ZooKeeper 集群在大规模元数据管理场景下的性能瓶颈问题，显著提升平台的稳定性。

迁移策略选择：为什么不做原地升级？

核心结论：原地升级 (In-place Upgrade) 风险大



元数据不兼容

v21与v24在 ZooKeeper 元数据格式和系统表结构上存在严重不兼容。



大量破坏性变更

3年代码差距积累大量底层变更，核心功能与API无法平滑过渡。



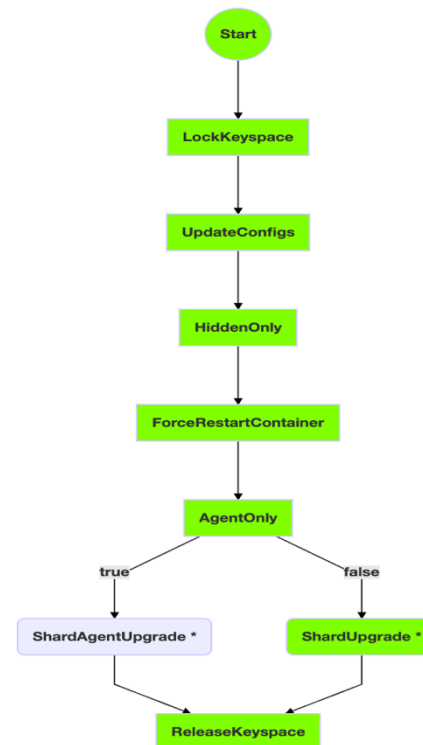
Rollback 极其困难

升级失败后回滚v21挑战极大，易导致长时间服务中断。



底层存储格式变更

压缩算法设置、Nullable 字段约束在两个版本间存在差异，直接升级会引发数据读取错误。



替换升级：逐分片Dual Consume

策略核心思路

对每个 Shard，新建一个隐藏的 v24 副本集，与旧的 v21 副本集并行运行。在完成流量切换前，新副本集对业务客户端完全透明，不产生任何影响。通过内部机制进行Dual Consume达成对用户无感的双写。



逐 Shard 执行

一次只迁移一个分片，将故障影响范围控制在最小粒度，最大程度降低风险。



替换式升级

不为每个 Shard 做原地升级，而是建立全新的 ZK 集群与 v24 副本集。



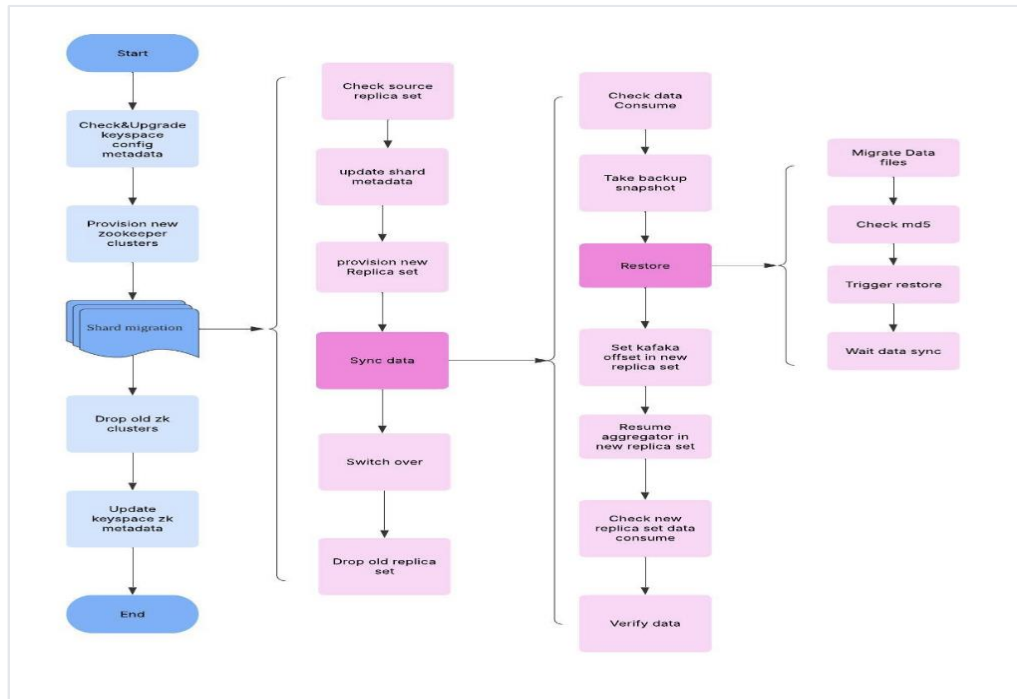
Dual Consume同步机制

利用backup tool迁移存量历史数据，同时通过 Kafka 实时同步增量数据，确保新旧数据最终一致性。



从数据构建元数据

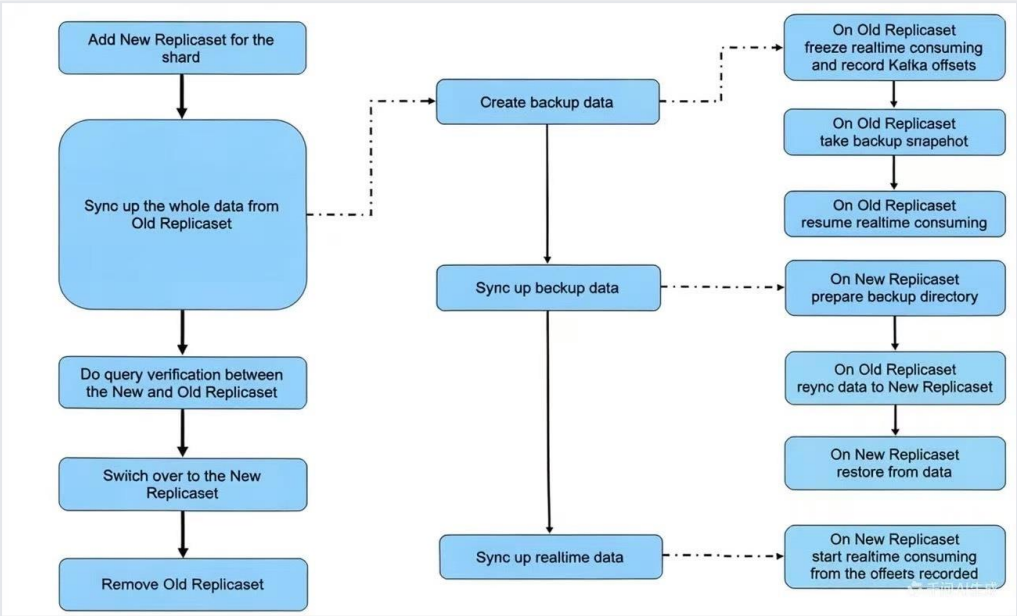
复制数据，不复制元数据，通过数据生成元数据



迁移流程全景图

旧集群→ 备份/双写 → 新集群→ 流量原子切换

分片迁移详细流程



准备阶段

为待迁移 Shard 创建全新 v24 副本集，创建与v21 集群中相同的表结构（Schema），作为数据迁移的目标端。



数据同步阶段

先使用clickhouse-backup tool同步旧数据快照，随后持续同步增量实时数据，确保新旧一致。



验证与切换

进行三重数据验证，确认无误后执行原子切换切至新副本集。



清理阶段

确认新副本集稳定后，删除旧 v21 副本集，释放硬件资源。

数据完整性保障：三层验证策略

01

历史数据验证

Historical Data Validation

■ **方法**：在校验备份文件的传输过程中，自动计算并校验其**MD5 Checksum**。

■ **目的**：确保海量的历史备份文件在跨网络传输和存储过程中没有被篡改或物理损坏。

02

实时数据验证

Realtime Data Validation

■ **方法**：持续监控新旧两个副本集的 Kafka Consumer Lag 指标。

■ **目的**：确认实时增量数据被正确、及时地写入新的 v24 副本集，无延迟堆积。

03

SQL 查询对比验证

SQL Query Comparison

■ **方法**：在新旧副本集上，对核心业务表执行相同的代表性查询（如 **COUNT(*), SUM(), AVG()**等）。

■ **目的**：通过直接对比统计结果，确保全量迁移完成后，新旧集群的数据状态完全一致。

数据完整性保障：备份快照关键步骤



核心挑战：在异步复制的架构下，如何创建一个时间点一致的、对用户无感知的备份？

01

冻结实时消费

临时暂停Kafka消费者，精准记录当前的消费Offset，确保备份数据不丢失、不重复。

02

清空Buffer并检查一致性

在ClickHouse内部强制清空写入缓冲区，并校验副本间的数据复制状态，确保快照源一致。

03

创建备份快照

确认集群所有数据处于强一致状态后，立即触发ClickHouse全量快照的生成流程。

04

恢复实时消费

快照创建指令下发完成后，即刻恢复Kafka消费者进程，将业务影响降至最低。



保障机制：对流程中的每个步骤均设置独立的超时自动恢复机制，避免异常情况导致服务长时间停顿。

完整的回滚预案



核心思想：为迁移过程中的每一个阶段都设计了对应的回滚路径，确保在任何情况下都能安全地恢复到初始状态。



场景一：Shard尚未切换

操作：直接删除新建的v24副本集及其相关资源。

影响：无任何数据操作，对业务无影响。



场景二：已切换(旧集存在)

操作：通过控制平面执行原子切换回滚，将流量切回旧的v21副本集。

影响：业务流量有一次反向切换，数据无丢失。



场景三：已切换(旧集已删)

操作：按照迁移的逆向流程，重建旧的v21副本集，并从新v24副本集进行备份恢复数据。

影响：恢复时间较长，但能确保最终数据可恢复。

规模化挑战与解决方案

01

元数据兼容性挑战

ZK 与系统表结构不兼容，底层差异明显

解决方案

为每个 Shard 创建全新的 ZK 集群和数据集群，并在 control plane 实现两套配置管理。复制数据，从数据构建元数据，不 copy 元数据。

02

海量资源约束挑战

典型场景：Ads 集群含 250 个 Shard，千级 Pod 规模

解决方案

实施“逐 Shard 迁移”策略。单个 Shard 迁移完成后，立即释放旧版 v21 资源，在有限的集群资源池中高效完成整体升级。

03

数据一致性挑战

异步复制机制下，Clickhouse 最终一致性

解决方案

在执行备份快照前，自动强制执行“复制一致性检查”流程，确保生成的备份数据逻辑完整、状态一致。

可扩展性提升：ZooKeeper扩容



01 / 核心痛点

Ads业务线规模庞大，拥有**250**个
Shard，但原先仅部署了**5**个 ZK 集群

这导致单集群平均承载高达**50**个
Shard。在高吞吐场景下，ZK 成为制
约系统性能的严重瓶颈。



02 / 扩容方案

利用系统整体架构升级的契机，针
对性优化资源配比，将 ZooKeeper
集群资源进行横向扩展。

集群数量扩展至

10个

(原配置：5个)



03 / 显著收益

- Before: 5 集群 × 50 Shards/集群
- After: 10 集群 × **25 Shards/集群**

✓ ZK写入压力直接减半

✓ 异常率显著下降

系统整体稳定性与可用性大幅提升

升级成果

01 / 已达成的核心目标



全量完成迁移

成功将72个keyspace全部从v21.8.3.44 平滑迁移至v24.4.1.2088版本，无遗漏。



保障服务连续性

业务读操作实现**零停机**，实时写入流量仅出现**分钟级延迟**，对业务影响极小。



数据完整性保障

全流程校验通过，确保数据迁移过程**零数据丢失**、**零数据重复**，数据准确无误。



系统性能提升

完成 ZK 集群扩容，消除性能瓶颈；升级后系统异常率显著下降，稳定性与处理能力双重提升。



解锁平台新能力

支持使用 Materialized Views (MV)、Projection 及增强 JSON 类型等新特性，为业务创新提供更强支持。

02 / 沉淀可复用的技术设计模式



跨版本不兼容 Metadata 变更处理方案



Keyspace 重分片

经验总结与思考

替换升级 vs 原地升级

核心决策回顾：

对比三年版本跨度，原地升级存在大量未知风险，不可行。最终选择替换升级方案，从根本上规避隐患。

逐Shard迁移（逐步推进）

策略与优势：

以Shard为单位分批迁移，有效分散变更与系统风险；利用迁移释放资源实现平滑过渡，避免“一刀切”的巨大风险。

dual consume + 原子切换

技术保障：

在新旧版本间通过dual consume达到对用户透明的双写以确保数据同步；切换瞬间对业务完全无感知，严格保障迁移过程的原子性与数据一致性。

多层数据验证体系

核心思想与实践：数据完整性不能仅靠单点检查。我们结合MDS校验、Checksum校验与SQL逻辑核对，构建起严密的“三道防线”，最终实现数据准确率100%的目标。

回滚方案与迁移方案同等重要

安全原则：必须为迁移的每一个阶段设计对应的回滚路径，确保在出现突发状况时能快速、安全地恢复到初始状态，这是整个项目不可或缺的“安全网”。

Q & A

感谢聆听！

THANK YOU FOR LISTENING